

Протокол

Автор: Андрій Білий: <https://github.com/bilyi1andrii>

Проект на GitHub: https://github.com/bilyi1andrii/soft_arch_hw_bilyi.git

Налаштування

У цьому завданні я буду використовувати Consul (HashiCorp). Для цього треба було повністю забрати попередній config_server та замінити його на консула. Спочатку я додав його ініціалізацію всередині docker-compose.yml:

```
consul:
  image: hashicorp/consul:1.15
  container_name: consul
  ports:
    - "8500:8500"
  command: "agent -dev -client=0.0.0.0"
```

Це просто створить невеличкий консул-сервер, який буде доступний і зсередини, і ззовні за портом 8500.

Реєстрація мікросервісів

Для цього я прописую для кожного наведену інформацію, щоб записати сервіс до консула (id, hostname, port і адреса консула)

```
environment:
  - SERVICE_ID=facade1
  - SERVICE_HOST=facade
  - SERVICE_PORT=8000
  - CONSUL_URL=http://consul:8500
```

Замість того щоб використовувати якісь вбудовані клієнтські бібліотеки, можна скористатися httpx і надіслати запит на реєстрацію прямо до консула:

```
registration_payload = {
  "ID": SERVICE_ID,
  "Name": "facade-service",
  "Address": SERVICE_HOST,
  "Port": SERVICE_PORT,
  "Check": {
    "HTTP": f"http://{SERVICE_HOST}:{SERVICE_PORT}/health",
    "Interval": "10s",
    "Timeout": "5s",
    "DeregisterCriticalServiceAfter": "1m"
  }
}
```

```

    }
    try:
        await
app.state.http_client.put(f"{CONSUL_URL}/v1/agent/service/register",
json=registration_payload)
        print(f"[FACADE] Registered {SERVICE_ID} with Consul",
flush=True)
    except Exception as e:
        print(f"[FACADE] Consul registration failed: {e}", flush=True)

```

Для цього потрібно підготувати правильний payload, де ми вказуємо id, ім'я сервісу, його адресу, порт і ендпоінт як перевірити його працездатність. (добре, що я зробив ще заздалегідь /health). Так робиться схожим чином до кожного з мікросервісів.

Після розгортання бачимо, що всі сервіси працюють:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
024cd580a465	logging-service:latest	"uv run fastapi run _"	18 seconds ago	Up 7 seconds		logging1
05793af02c88	logging-service:latest	"uv run fastapi run _"	18 seconds ago	Up 7 seconds		logging3
4119458423d5	logging-service:latest	"uv run fastapi run _"	18 seconds ago	Up 7 seconds		logging2
fad27473b672	facade-service:latest	"uv run fastapi run _"	18 seconds ago	Up 11 seconds	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	facade
092e4f44dc2	counter-service:latest	"uv run fastapi run _"	18 seconds ago	Up 11 seconds		counter_service
6a0fbedeb27	postgres:15-alpine	"docker-entrypoint.s_"	18 seconds ago	Up 17 seconds (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp	postgres_db
7e287e39fc18	apache/kafka:latest	"w/_cacert_entrypoint_"	18 seconds ago	Up 17 seconds (healthy)	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp	kafka
c4ead4758e76	hashicorp/consul:1.15	"docker-entrypoint.s_"	18 seconds ago	Up 17 seconds	0.0.0.0:8500->8500/tcp, [::]:8500->8500/tcp	consul
dd829d258c3	hazelcast/management-center:5.3	"bash ./bin/mc-start_"	18 seconds ago	Up 17 seconds	0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp	management-center
bf013572541f	hazelcast/hazelcast:5.3	"hz start"	18 seconds ago	Up 17 seconds (healthy)	0.0.0.0:5703->5701/tcp, [::]:5703->5701/tcp	hazelcast3
ad86b8812791	hazelcast/hazelcast:5.3	"hz start"	18 seconds ago	Up 17 seconds (healthy)	0.0.0.0:5702->5701/tcp, [::]:5702->5701/tcp	hazelcast2
f9d375db682a	hazelcast/hazelcast:5.3	"hz start"	18 seconds ago	Up 17 seconds (healthy)	0.0.0.0:5701->5701/tcp, [::]:5701->5701/tcp	hazelcast1

А в Consul UI бачимо, що всі мікросервіси зареєструвалися і є правильна кількість інстансів у logging.

dc1

Help Settings

Overview

Services

Nodes

Key/Value

Intentions

ACCESS CONTROLS

Tokens

Policies

Roles

Auth Methods

ORGANIZATION

Peers

Services 4 total

Search

Search Across

Health Status

Service Type

Unhealthy to Healthy

consul

1 instance

counter-service

1 instance

facade-service

1 instance

logging-service

3 instances

Звертання facade-service

Щоб уникнути статичних адрес під час звертання до counter чи logging інстансів, я переважористовую попередню функцію, яка тепер базується на консулі, щоб отримати адресу та порт за service_name. За допомогою цього endpoint я також автоматично

перевіряю чи інстанс доступний, і вертаю лише ті, в яких `passing=true` (тобто, вони в ресурсі).

```
async def get_service_urls(http_client: httpx.AsyncClient, service_name:
str) -> list:
    try:
        resp = await http_client.get(

f"{CONSUL_URL}/v1/health/service/{service_name}?passing=true"
        )
        if resp.status_code == 200:
            instances = resp.json()
            urls = []
            for instance in instances:
                address = instance["Service"]["Address"]
                port = instance["Service"]["Port"]
                urls.append(f"http://{address}:{port}")
            return urls
        except Exception as e:
            print(f"[FACADE] Error fetching {service_name} from Consul:
{e}")
        return []
```

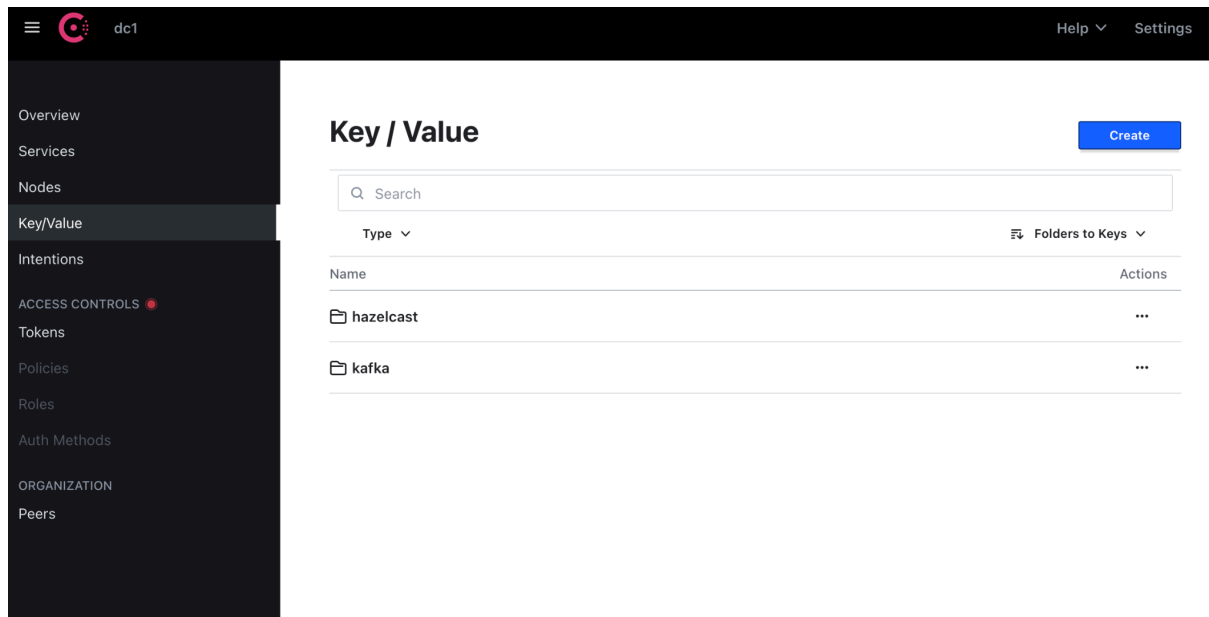
Hazelcast і Message Queue

Для того щоб власноруч не вказувати конфігурацію для Hazelcast та Kafka, я додав також невеличкий `consul-init`:

```
consul-init:
  image: hashicorp/consul:1.15
  container_name: consul-init
  depends_on:
    - consul
  environment:
    - CONSUL_HTTP_ADDR=http://consul:8500
  command: >
    sh -c "
      sleep 2 &&
      consul kv put hazelcast/config
      '{\"cluster_name\": \"dev-cluster\", \"members\": [\"hazelcast1:5701\", \"ha
zelcast2:5701\", \"hazelcast3:5701\"]}' &&
      consul kv put kafka/config
      '{\"brokers\": \"kafka:9092\", \"topic\": \"transactions\"}'
    "
```

Загалом, під час ініціалізації, він звертатиметься до консула за адресою `http://consul:8500` всередині мережі Docker і додасть потрібні конфігурації для мапи

(назва кластеру, адреси його учасників) та черги (її адреса та назва топіку). В UI вони зберігаються так:



Ці дані будуть відповідно діставатися в коді logging та counter сервісів:

logging

```
try:
    kv_resp = await http_client.get(
        f"{CONSUL_URL}/v1/kv/hazelcast/config?raw=true"
    )
    hz_config = kv_resp.json()
    print(f"[LOGGING] Fetched Hazelcast Config: {hz_config}",
flush=True)
except Exception as e:
    print(f"[LOGGING] Failed to fetch Hazelcast config: {e}",
flush=True)
    hz_config = {"cluster_name": "dev-cluster", "members":
["hazelcast1:5701"]}
```

counter

```
try:
    kv_resp = await
http_client.get(f"{CONSUL_URL}/v1/kv/kafka/config?raw=true")
    kafka_config = kv_resp.json()
    kafka_topic = kafka_config.get("topic", "transactions")
    kafka_brokers = kafka_config.get("brokers", "kafka:9092")
    print(f"[COUNTER] Fetched Kafka Config: {kafka_config}",
flush=True)
except Exception as e:
```

```
print(f"[COUNTER] Failed to fetch Kafka config: {e}", flush=True)
kafka_brokers = "kafka:9092"
kafka_topic = "transactions"
```

Якщо в разі не вдається доступитися до консула (сталася помилка), то я даю якусь типову конфігурацію (про всяк).

Можете побачити як сервіси дістають дані з консула:

```
[LOGGING] Fetching Hazelcast Config: {'cluster_name': 'dev-cluster', 'members': ['hazelcast1:5701', 'hazelcast2:5701', 'hazelcast3:5701']}
[LOGGING] Connected to Hazelcast cluster!
[LOGGING] Registered logging1 with Consul
INFO Application startup complete.
INFO Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO 172.18.0.6:40720 - "GET /health HTTP/1.1" 200
INFO 172.18.0.10:38312 - "GET /health HTTP/1.1" 200
INFO 172.18.0.6:47652 - "GET /health HTTP/1.1" 200
INFO 172.18.0.10:59136 - "GET /health HTTP/1.1" 200
INFO 172.18.0.6:48544 - "GET /health HTTP/1.1" 200
```

```
[COUNTER] Successfully connected to PostgreSQL!
[COUNTER] Fetched Kafka Config: {'brokers': 'kafka:9092', 'topic': 'transactions'}
[COUNTER] Registered with Consul
INFO Application startup complete.
INFO Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

[COUNTER] Kafka Consumer started listening to 'transactions' topic
INFO 172.18.0.6:34668 - "GET /health HTTP/1.1" 200
INFO 172.18.0.10:34614 - "GET /health HTTP/1.1" 200
INFO 172.18.0.6:39904 - "GET /health HTTP/1.1" 200
INFO 172.18.0.10:54532 - "GET /health HTTP/1.1" 200
INFO 172.18.0.6:41810 - "GET /health HTTP/1.1" 200
INFO 172.18.0.10:53466 - "GET /health HTTP/1.1" 200
INFO 172.18.0.6:57904 - "GET /health HTTP/1.1" 200
```

Також можна побачити стабільні перевірки на health, це якраз прописувалось у payload до консула (ключ Check)

Відмовостійкість

Звичайна Імплементация

Зроблю 5 тестових запитів на користувача test.

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "test",
  "amount": 10
}'
```

Request URL

```
http://localhost:8080/process
```

Server response

Code	Details
200	Response body

```
{
  "transaction_id": "1777812667549654046",
  "status": "Message queued successfully"
}
```

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "test",
  "amount": 12
}'
```

Request URL

```
http://localhost:8080/process
```

Server response

Code	Details
200	Response body

```
{
  "transaction_id": "1777812691524678335",
  "status": "Message queued successfully"
}
```

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "test",
  "amount": -32
}'
```

Request URL

```
http://localhost:8080/process
```

Server response

Code	Details
200	Response body

```
{
  "transaction_id": "1777812714491070846",
  "status": "Message queued successfully"
}
```

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "test",
  "amount": 40
}'
```

Request URL

```
http://localhost:8080/process
```

Server response

Code	Details
200	Response body

```
{
  "transaction_id": "1777812739194527552",
  "status": "Message queued successfully"
}
```

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "test",
  "amount": 100
}'
```

Request URL

```
http://localhost:8080/process
```

Server response

Code	Details
200	Response body

```
{
  "transaction_id": "1777812754927943628",
  "status": "Message queued successfully"
}
```

Перевірка за GET

Curl	<pre>curl -X 'GET' \ 'http://localhost:8080/user/test' \ -H 'accept: application/json'</pre>
Request URL	<pre>http://localhost:8080/user/test</pre>
Server response	
Code	Details
200	Response body <pre>"balance": 130, "transactions": [{ "user_id": "test", "amount": 100, "transaction_id": "1777812754927943628" }, { "user_id": "test", "amount": 10, "transaction_id": "1777812667549654046" }, { "user_id": "test", "amount": 40, "transaction_id": "1777812739194527552" }, { "user_id": "test", "amount": 12, "transaction_id": "1777812691524678335" }, { "user_id": "test", "amount": -32,</pre>

```
}
  "transaction_id": "1777812714491070846"
}
```

А тепер зупиню перший інстанс логування:

```
docker pause logging1
```

Бачимо, що сервіс на UI недоступний:

logging-service

Instances	Intentions	Routing	Tags
logging1			
❌ All service checks failing ✅ All node checks passing ↻ dce32b6064d5 🌐 logging1:8000			
logging2			
✅ All service checks passing ✅ All node checks passing ↻ dce32b6064d5 🌐 logging2:8000			
logging3			
✅ All service checks passing ✅ All node checks passing ↻ dce32b6064d5 🌐 logging3:8000			

На /health endpoint та сама історія:

Curl

```
curl -X 'GET' \
  'http://localhost:8080/health' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/health
```

Server response

Code	Details
200	Response body <pre>{ "status": "ok", "consul": "ok", "kafka": "ok", "counter_services": { "http://counter_service:8000": { "status": "ok", "database": "connected" } }, "logging_services": { "http://logging1:8000": "down", "http://logging2:8000": { "status": "ok", "hazelcast": "connected" }, "http://logging3:8000": { "status": "ok", "hazelcast": "connected" } } }</pre>

Зробимо два тестових запити на test2:

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "test2",
  "amount": 100
}'
```

Request URL

```
http://localhost:8080/process
```

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1777813642623063928", "status": "Message queued successfully" }</pre>

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "test2",
  "amount": -50
}'
```

Request URL

```
http://localhost:8080/process
```

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1777813663358614466", "status": "Message queued successfully" }</pre>

Бачимо з логів, що запити перенаправлялись на залишені інстанси (також відбувається постійна перевірка здоров'я):

logging2

d6d6eb0653b2

logging-service:latest

STATUS

Running (18 minutes ago)

Logs	Inspect	Bind mounts	Exec	Files	Stats
INFO	172.18.0.6:41438	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:38234	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:43962	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:40842	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:54976	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:41688	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:56710	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:54774	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:48588	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:52418	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:54228	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:52914	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:39504	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:52922	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:35860	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:35188	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:48124	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:44834	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:35086	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:60668	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:59684	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:43482	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:54952	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:37344	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:55326	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:36478	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:59696	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:60234	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:45560	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:41738	-	"GET /health HTTP/1.1"	200	
[LOGGING]	Stored transaction 1777813663358614466 for user 'test2'				
INFO	172.18.0.10:41738	-	"POST /transaction HTTP/1.1"	200	

logging3

786f433c84fe

logging-service:latest

STATUS

Running (18 minutes ago)

Logs	Inspect	Bind mounts	Exec	Files	Stats
INFO	172.18.0.10:38212	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:39208	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:59552	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:45406	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:53410	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:43888	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:33792	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:37360	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:59098	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:60088	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:35050	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:45800	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:56924	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:33264	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:58446	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:40196	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:42622	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:60180	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:54596	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:34482	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:60150	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.6:47580	-	"GET /health HTTP/1.1"	200	
INFO	172.18.0.10:38636	-	"GET /health HTTP/1.1"	200	
[LOGGING]	Stored transaction 1777813642623063928 for user 'test2'				
INFO	172.18.0.10:38636	-	"POST /transaction HTTP/1.1"	200	

Перевірка за GET

Curl

```
curl -X 'GET' \
'http://localhost:8080/user/test2' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:8080/user/test2
```

Server response

Code	Details
200	Response body <pre>{ "balance": 50, "transactions": [{ "user_id": "test2", "amount": 100, "transaction_id": "1777813642623063928" }, { "user_id": "test2", "amount": -50, "transaction_id": "1777813663358614466" }] }</pre>

Curl

```
curl -X 'GET' \
'http://localhost:8080/accounts' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:8080/accounts
```

Server response

Code	Details
200	Response body <pre>{ "test": 130, "test2": 50 }</pre>

Тестування Продуктивності

Для цього використовую той самий скрипт з першого домашнього завдання.

```
uv run perf_test.py
```

```
=====
Starting Scenario 1 (10 Clients, 10 Unique Accounts)
=====

--- Performance Results ---
Total Requests: 100000
Total Time: 181.77 seconds
Requests Per Second (RPS): 550.14

--- Internal Time Contributions ---
Logging Service Total Delay: 1303.47s
Counter Service Total Delay: 674.06s

--- Balance Verification ---
user_0 Balance: 10000.0 (Expected: 10000.0)
user_1 Balance: 10000.0 (Expected: 10000.0)
user_2 Balance: 10000.0 (Expected: 10000.0)
user_3 Balance: 10000.0 (Expected: 10000.0)
user_4 Balance: 10000.0 (Expected: 10000.0)
user_5 Balance: 10000.0 (Expected: 10000.0)
user_6 Balance: 10000.0 (Expected: 10000.0)
user_7 Balance: 10000.0 (Expected: 10000.0)
user_8 Balance: 10000.0 (Expected: 10000.0)
user_9 Balance: 10000.0 (Expected: 10000.0)
```

```
=====
Starting Scenario 2 (10 Clients, 1 Shared Account)
=====

--- Performance Results ---
Total Requests: 100000
Total Time: 188.15 seconds
Requests Per Second (RPS): 531.49

--- Internal Time Contributions ---
Logging Service Total Delay: 1348.21s
Counter Service Total Delay: 695.88s

--- Balance Verification ---
Shared Account Balance: 100000.0 (Expected: 100000.0)
```

Також забув показати скріншот які логи всередині фасаду відбуваються, то прикріплюю їх тут:

```

[FACADE] Received request from user 'shared' for amount 1.0
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:22626 - "POST /process HTTP/1.1" 200
INFO    192.168.65.1:36140 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:27199 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:55775 - "POST /process HTTP/1.1" 200
INFO    192.168.65.1:55881 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:63354 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'shared' for amount 1.0
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:40043 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:47019 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:38547 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:50666 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:22626 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:36140 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'shared' for amount 1.0
INFO    192.168.65.1:27199 - "POST /process HTTP/1.1" 200

```

Табличка Порівняння

Test scenarios	Task 1 (in-mem)	Task 3 (DB)	Task 5 (final)
10 accounts	173.96s	200.79s	181.77s
	1014.24s	493.78s	1303.47s
	991.39s	1556.05s	674.06s
1 account	174.48s	205.22s	188.15s
	1011.81s	525.40s	1348.21s
	993.84s	1597.94s	695.88s

Почнемо зі загального часу виконання, можна побачити, що коли в нас нема бази даних, черги повідомлень і менше зайвих запитів мережею, то ми отримуємо майже 174 секунди виконання на обидвох кейсах. Далі, в наступній домашці ми додали дві речі: декілька інстансів сервісу логування та реляційну базу даних для збереження

балансу користувача. Хоч ми і отримали постійне, довгострокове та більш масштабоване зберігання даних (не в RAM), але в той самий час ми тепер робимо додаткові запити до postgres, які додають ще більшої затримки (через мережу). Навіть маючи тепер швидкісну distributed map, ми все рівно мусимо чекати на найповільнішу ланку системи, тому загальний час збільшився до приблизно 200 секунд. І на кінець, в останній версії системи з'явилася черга повідомлень (Kafka) та Consul. За допомогою першої ми змогли виправити недоліки звертань до бази, роблячи їх закадрово. Тому загальний час зменшився до 181 секунди (про консула згодом). Можна зауважити що загальний час на другий сценарій був більшим у всіх випадках імплементації. Напевно це через те, що ми оновлюємо значення балансу одного і того самого користувача і база витрачає більше часу на забезпечення синхронізації.

Тепер поговоримо про те, як змінювалась контрибуція сервісу розрахунку балансу. Як ми вже знаємо, in-memory база є значно швидшою ніж дискова через повільність I/O операцій та мережі. Тому час значно збільшився у третьому домашньому (з 991s до 1556s). Але коли ми додали чергу повідомлень, то ми змогли нівелювати синхронні запити до бази через тепер непряме звертання до counter сервісу. Таким чином, ми тепер спілкуємось з чергою повідомлень, і чекаємо лише на те, щоб дані записалися до топіку, а далі вже counter consumer зробить своє діло. Як результат час зменшився до 674 секунд (відповідно на другому сценарію він трішки більший).

І накінець як же ж змінювався тренд часу для сервісу логування. На початку ми мали один екземпляр, що використовував in-memory хеш-таблицю, яка дуже швидко зростала і відповідно погано масштабувалася. Щоб це подолати, ми зробили невеличкий load balancing (тепер 3 екземпляри) та використали Hz distributed map для швидкого зберігання логів. Як наслідок, час зменшився з 1014 до 493 секунд. Але в 5 лабораторній, ми бачимо що час зріс на значно вище ніж було. Я думаю що це пов'язано з консулом, адже тепер перед тим як робити запит до logging, ми маємо запитатися (по мережі) нашого консула, які екземпляри є доступними на даний момент. А якщо це робити на 100к запитів, то це дуже швидко стане вузьким місцем. Тому час зріс майже до 1304 секунд. Щоб це виправити ймовірно треба додати деякий елемент кешування, наприклад Redis, де ми будемо зберігати інформацію про існуючі інстанси і оновлювати її що 5 хвилин.

В загальному, система получилась цікава і не надто bloated. В ній дуже легко тепер промасштабувати кожен з мікросервісів; менше потрібно переживати за data persistence, адже ми маємо реляційну базу та потрібну реплікацію на Hz нодах; за допомогою консула легше моніторити стан здоров'я кожного з мікросервісів, і не потрібно хардкодити конфігурацію допоміжний сервісів, як черга повідомлень чи Hz кластер, але звичайно тепер він стає таким собі single point of failure, що не завжди бажано.